Università degli studi di Padova
Facoltà di Ingegneria

Introduzione a LabVIEW

M.Bertocco

1

LabVIEW

(Laboratory Virtual Instrument Engineering Workbench)

Ambiente di sviluppo per applicazioni, in particolare
- acquisizione dati e gestione strumentazione
- analisi di segnali

Ambiente interattivo di programmazione grafica e a oggetti
- programmazione = *G-language* = disegno di diagrammi a blocchi

Prevede integrazione con sistemi per acquisizione dati e controllo di strumentazione
- interfacce IEEE-488, RS-232/422, VXI, schede A/D-D/A …

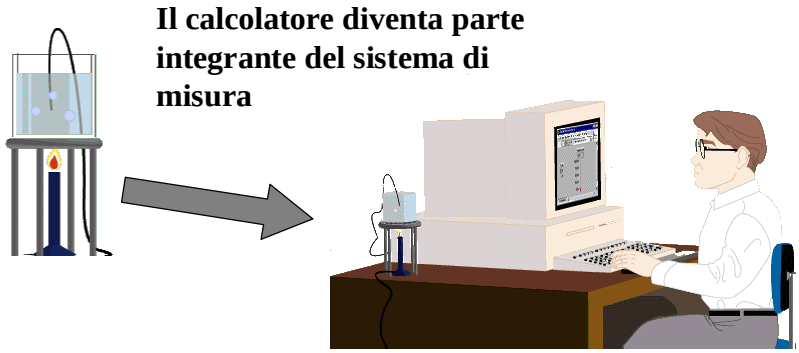
Ricca collezione di librerie
- interazione con S.O. (file i/o, DLL, ActiveX, …)
- grafica, analisi dei segnali, ...

2

Introduzione a LabVIEW

Motivazione principale
controllo di strumentazione da calcolatore

esempio: misura di temperatura



Il calcolatore diventa parte integrante del sistema di misura

3

Introduzione

Principali differenze tra strumento stand-alone e strumento controllato da calcolatore

strumento



- interazione attraverso il pannello frontale
- pannello frontale fisso
- funzioni svolte da strumento prefissate

---

calcolatore



- interazione indiretta con più strumenti attraverso mouse, tastiera, touch-screen,...
- pannello frontale dipendente da programma
- possibiltà di elaborazione dei dati o di coordinare le azioni di più strumenti

4

Introduzione

Il calcolatore mostra un insieme di strumenti (reali) come un unica entità diversa dalle singole sub-unità fisiche che compongono la stazione di test

→ **strumento virtuale (VI)**

esempio

- strumenti “reali” a disposizione: generatore di funzioni + oscilloscopio + calcolatore
- programma (VI) che determina e mostra direttamente la risposta in frequenza di un circuito lineare

- strumento virtuale = misuratore di risposta in frequenza

5

Introduzione

In LabVIEW un programma è detto **Virtual instrument (VI)**

LV è un ambiente grafico di programmazione nel quale il codice sorgente è molto simile a un diagramma di flusso (flow-chart)

- Ogni nodo del diagramma di flusso esegue una funzione specifica

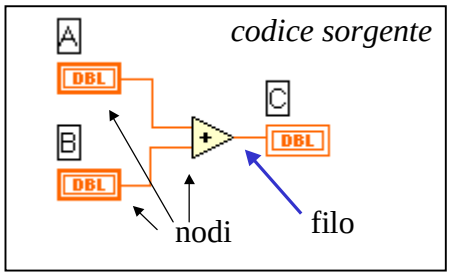
- I nodi sono connessi tra loro da fili

- L'ordine di esecuzione dei nodi è fissato dal flusso di dati che fluisce da un nodo ai successivo

6

Introduzione

esempio: somma tra due numeri



"somma.vi" :
esegue la somma di "A"+ "B", e deposita in "C" il risultato

- *i dati fluiscono* :
  - dai nodi A, B verso l'icona "+"
  - dal nodo di somma verso "C"

- *ordine di esecuzione* :
  - solo quando all'ingresso del nodo di somma entrambi i dati forniti da "A" e "B" sono disponibili viene eseguita la somma
  - solo quando il nodo di somma rende il risultato disponibile, "C" permette la rappresentazione a video

7

Programma LabVIEW

programma LabVIEW → **V*irtual Instrument* (*VI*)**

si compone di **tre parti**:

1. pannello frontale
2. diagramma a blocchi funzionale
3. icona e connettore

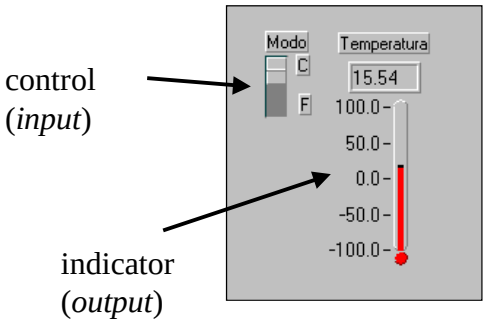
8

Componenti di un Virtual Instrument

1. Pannello frontale

permette l'interazione con l'utente

- elementi di ingresso *controlli* (manopole, bottoni,…)
- elementi di uscita *indicatori* (spie, dati numerici grafici, …)



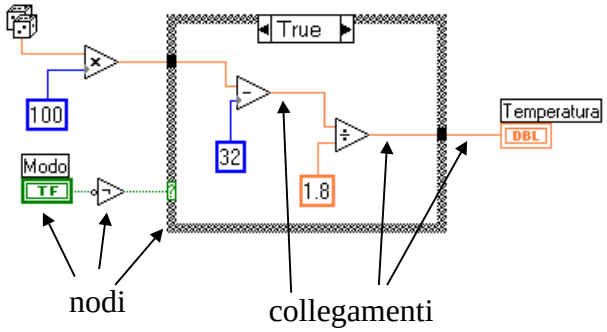
9

Componenti di un Virtual Instrument

2. Diagramma a blocchi

contiene il codice nella forma di un diagramma a blocchi, ed è costituito da:

- *nodi* : elementi di elaborazione elementare
- *collegamenti* : uniscono i nodi e permettono lo scambio di dati

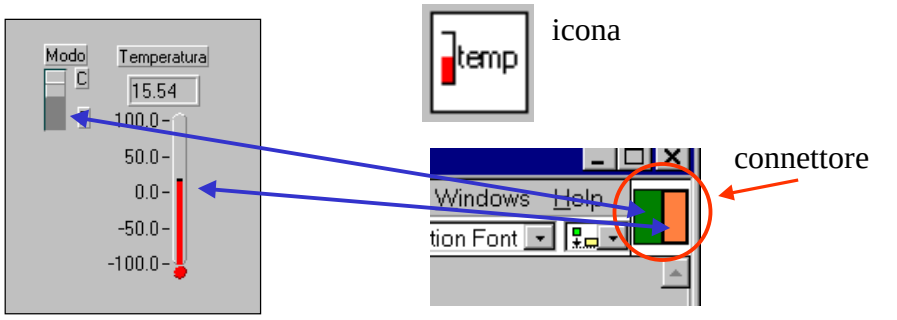


10

Componenti di un Virtual Instrument

3. Icona e connettore

- permette di trasformare un programma in un oggetto
- i terminali determinano la corrispondenza tra aree dell'icona e controlli/indicatori del pannello frontale



pannello frontale

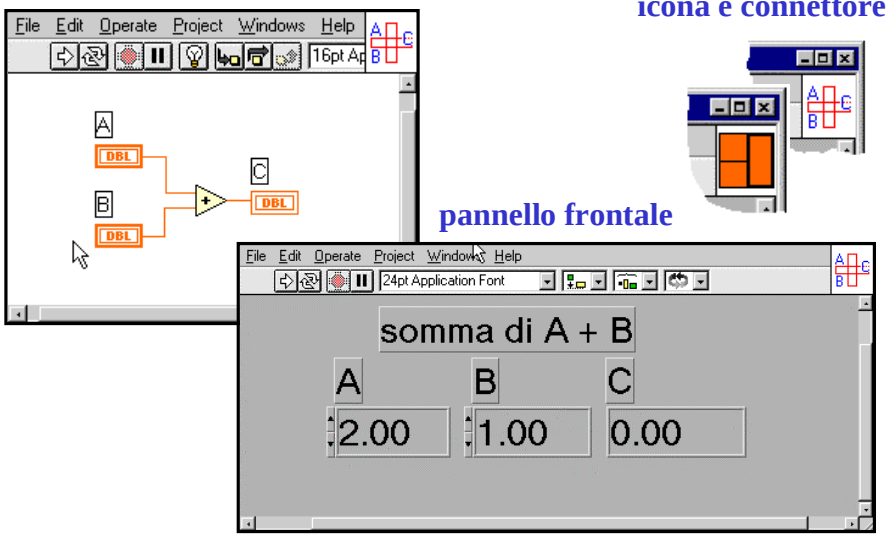
corrispondenza terminali - elementi pannello frontale (ottenuta realizzando dei collegamenti)

11

Componenti di un Virtual Instrument

diagramma a blocchi

icona e connettore



12

PROGRAMMA

- codice
- dati
- flusso di elaborazione

programma "tradizionale"	*programma LabVIEW*
CODICE	
• collezione di moduli sorgente • istruzioni nella forma di un testo	• collezione di VI • diagramma a blocchi
DATI	
• contenuti in variabili di programma	• contenuti in oggetti (VI)

13

programma "tradizionale"	*programma LabVIEW*
ELABORAZIONE	
• le istruzioni modificano il contenuto delle variabili	• oggetti (VI) passano informazioni ad altri oggetti tramite i collegamenti
• istruzioni eseguite in sequenza • costrutti "di controllo" possono alterare l'ordine di esecuzione delle istruzioni	• esecuzione parallela • ordine di esecuzione data da: • disponibilità informazioni all'ingresso di un oggetto • strutture "di controllo" (funzionalità simile linguaggio tradizionale)
MODULARITA`	
• definizione di task • task → moduli, classi, funzioni	• definizione di task • task → VI

14

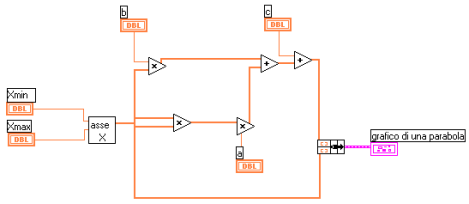


ALCUNI ESEMPI

programma "tradizionale" ***programma LabVIEW***

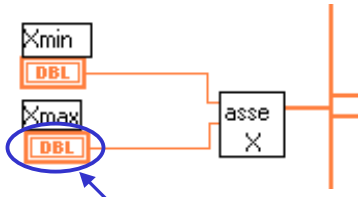
file "parabola.c" **CODICE** *file "parabola.vi"*

```
...
for (i=0;i<MAX;i++)
   x[i] = min+delta*i
for (i=0;i<MAX;i++)
   y[i] = a*x[i]*x[i]+b*x[i]
...
```



DATI

```
#define MAX 100
int i;
float x[MAX], y[MAX];
...
```



contiene la variabile "Xmax"

15

ALCUNI ESEMPI

programma "tradizionale" ***programma LabVIEW***

DATI

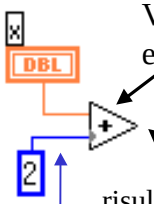
```
float y,x;

y = x+2;
```

dichiarazione delle variabili impiegate

significato:

alla variabile y viene assegnato il risultato della somma tra il valore della var. x e la costante 2



VI "addizione" esegue somma

risultato operazione disponibile all'uscita del VI "addizione"

trasferimento del dato costante "2" al VI che esegue la somma

16

ALCUNI ESEMPI

programma "tradizionale" ***programma LabVIEW***

ELABORAZIONE: sequenza

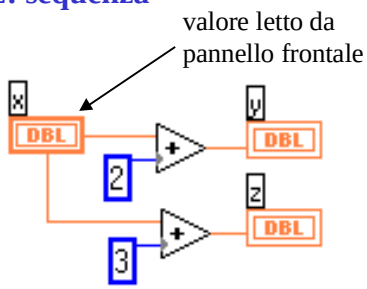
```
float y,x,z;
...
scanf("%f", &x);
y = x+2;
z = x+3;
```

x	y	z
1	?	?
1	3	?
1	3	4

flusso di elaborazione

"lettura di x" da tastiera

istruzioni eseguite in sequenza



esecuzione parallela

- i nodi di somma forniscono simultaneamente il risultato
- y, z aggiornate contemporaneamente

17

ALCUNI ESEMPI

programma "tradizionale" ***programma LabVIEW***

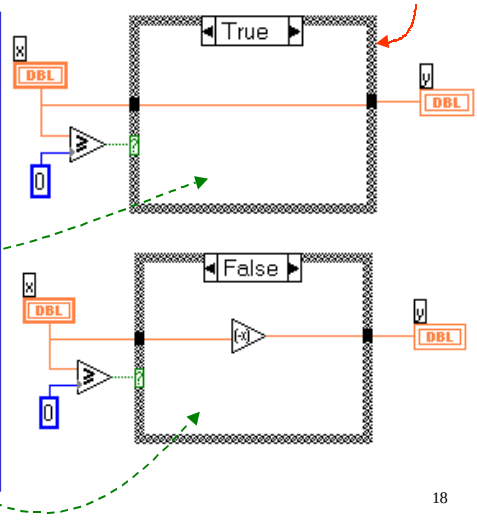
ELABORAZIONE: strutture di controllo

"if"

esempio: "if - then - else"

y = |x|

```
if (x >= 0)
   y = x
else
   y = -x
```



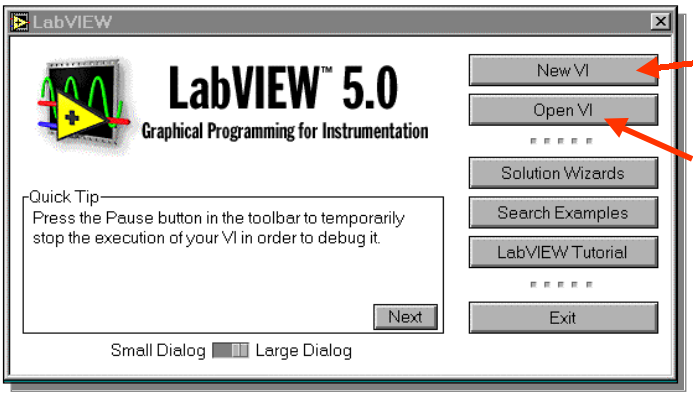
18



Ambiente di lavoro

Avvio di LabVIEW

nuovo Virtual Instrument (programma)



apre VI già esistente

19

Ambiente di lavoro

costruzione di un VI

- costruzione pannello frontale (front panel)
- costruzione connettore (connector) e icona (icon)
- costruzione codice (data diagram)

esecuzione e debug di un VI

- run: interazione solo con pannello frontale
- debug: interazione con pannello frontale e block diagram

20



Costruzione di un VI

pannello frontale

consente di predisporre l'interfaccia utente

- si dispongono (e configurano) gli elementi per l'interazione con l'utente mediante operazioni di "drag & drop" con il mouse
- elementi disponibili ottenuti da una lista ("controls")

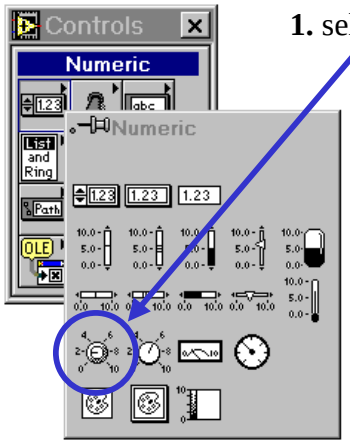


numeric, boolean, string&table
list&ring, array&cluster, graph
path & refnum, decorations, user controls
activeX, custom control

ricca collezione di elementi predefiniti
possibilità di crearne di nuovi (user-defined)

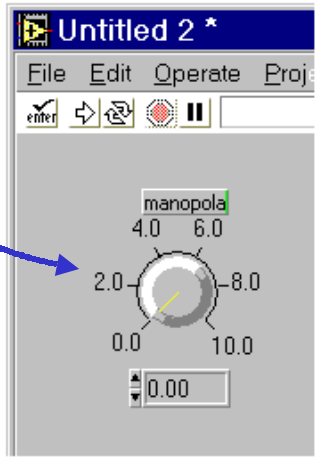
21

esempio: inserzione di una manopola



1. selezione elemento "manopola"

2. "drag& drop" su finestra front panel



- eventuale operazione di ridimensionamento dell'elemento grafico
- azioni di posizionamento degli elementi grafici o di impostazione font mediante *menu toolbar* del pannello frontale

22

Costruzione di un VI

pannello frontale

- drag & drop di indicatori e controlli (control toolbar)
- modifica elementi mediante *popup* (click destro) su elemento, o azioni con mouse (tools palette)



 *operating tool* - modifica valore elemento

 *positioning tool*: selezione, resize

 *labeling tool*: edizione testo, creazione label

 *object popoup tool*: attiva menu popup

 *scroll tool*: scroll finestra senza usare barre

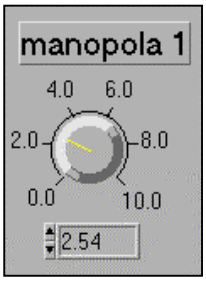
 *color copy tool*: copia colori

 *color tool*: impostazione colori

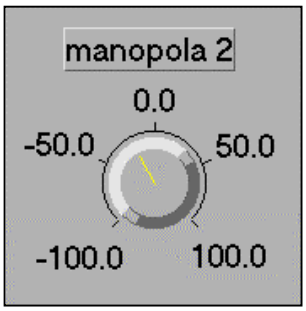
23

un semplice esempio

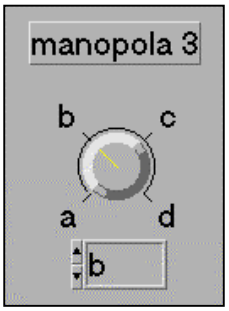
control di tipo “knob” (manopola) in tre versioni distinte



manopola di default



- assenza indicatore numerico
- fondo scala = 100



valori possibili di ingresso in un insieme finito, e associati alle indicazione (a, …, d)

24

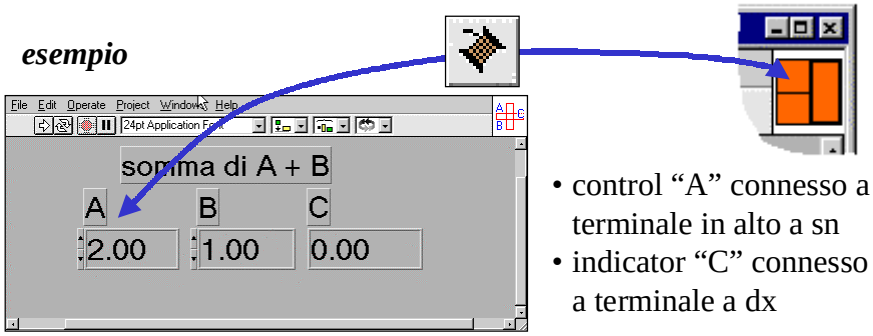
Costruzione di un VI

Connettore

- popup su area icona del front panel
- menu *show connector*

  l'icona si trasforma in un'area a rettangoli
- selezionare rocchetto (tools palette)
- collegare (click) gli elementi del pannello frontale con aree distinte del connettore (di solito a sn controls, a dx indicators)

esempio



- control "A" connesso a terminale in alto a sn
- indicator "C" connesso a terminale a dx

Costruzione di un VI

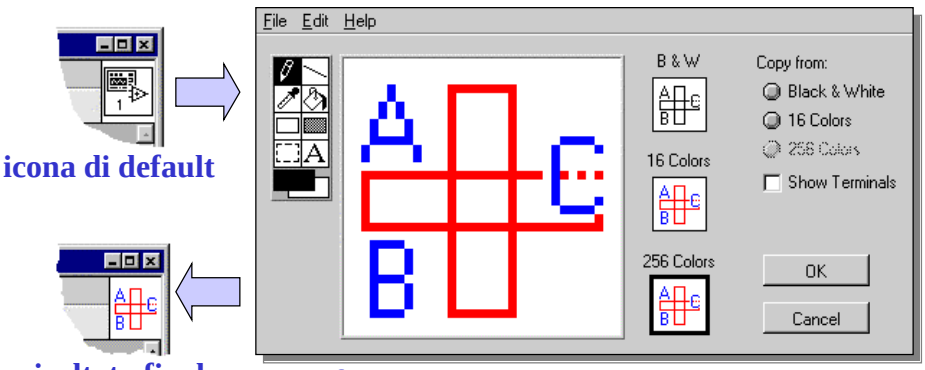
Icona

- popup su area icona del front panel
- menu *edit icon*

  compare finestra di dialogo per edizione dell'icona

oppure

- drag & drop di una bitmap su area icona



icona di default

risultato finale

finestra di dialogo per edizione icona



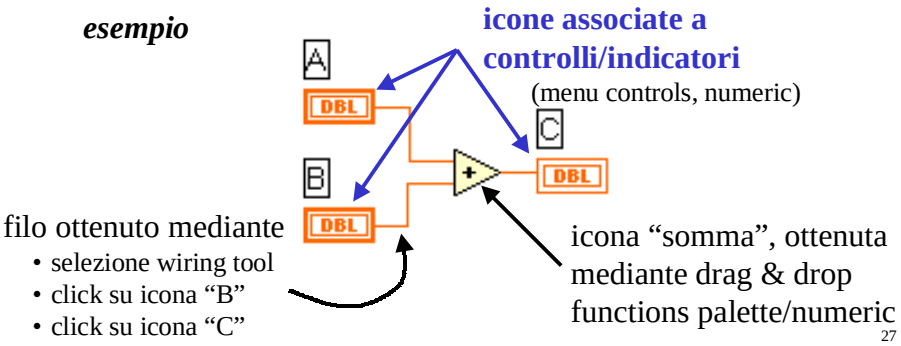
Costruzione di un VI

block diagram

- drag & drop di funzioni (icone) da *function palette*
- connessione icone mediante *wiring tool* (tools palette) 
  - click su prima icona, spostamento rocchetto, click su seconda icona, eventuali click intermedi nel caso di collegamenti più complessi
  - uso di "positioning tool" per modifica collegamenti, posizione icone,...
- menu *edit/remove bad wires* 

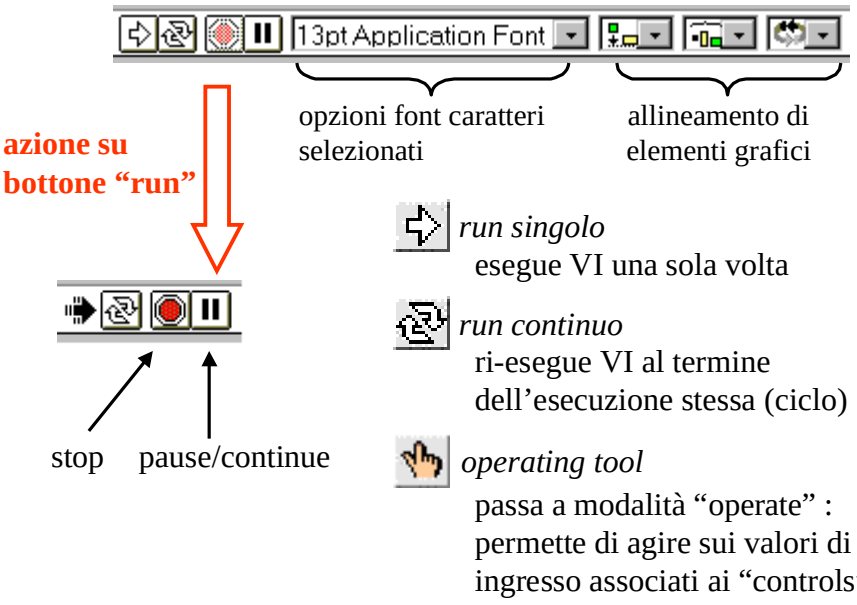
---

esempio



27

Esecuzione di un VI - *front panel toolbar*



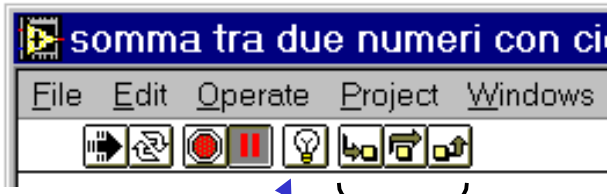
run singolo
esegue VI una sola volta

run continuo
ri-esegue VI al termine dell'esecuzione stessa (ciclo)

operating tool
passa a modalità "operate" : permette di agire sui valori di ingresso associati ai "controls"

28

Debug di un VI - *block diagram toolbar*



come toolbar front panel, in più:

DEBUG

esecuzione "single step"
esegue una singola "istruzione" e poi ferma esecuzione

- apre sub-VI
- esegue sub-VI senza aprirlo
- esce da sub-VI

highlight execution
se opzione attiva, l'ambiente mostra una animazione nella quale i dati in transito sui connettori sono rappresentati da delle palline in movimento
→ comprensione sequenza operazioni svolte dal VI

29

Debug di un VI - *block diagram toolbar*





probe

una volta selezionato lo strumento *probe*, se seleziona (click) un filo compare una finestra che mostra il contenuto della variabile che scorre lungo quel filo



breakpoint

una volta selezionato lo strumento *breakpoint*, se si seleziona (click) un filo compare una icona che rappresenta un segnale di stop. Quando un dato transita in quella posizione viene sospesa l'esecuzione del VI

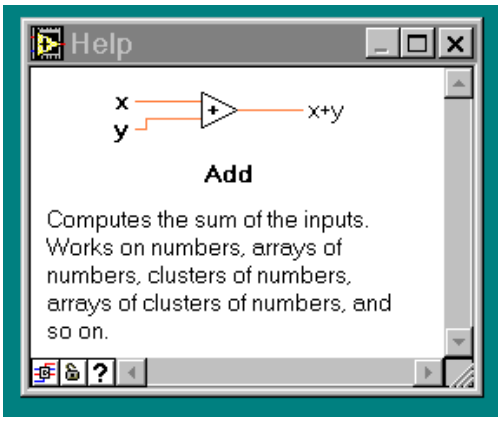
30

On-line help

- visualizzazione finestra di documentazione associata ad un VI
- documentazione predisposta dal programmatore per “nuovi” VI (front panel menu *window / show VI info*)

esempio

quando il mouse è sovrapposto a un VI di tipo “addizione”, presente sul block diagram



Sub-VI

VI utilizzato all’interno del VI corrente

sub-VI mostrato come icona nel block diagram del VI corrente

terminali collegati all’icona come definito dal connettore del sub-VI

---

esempio:

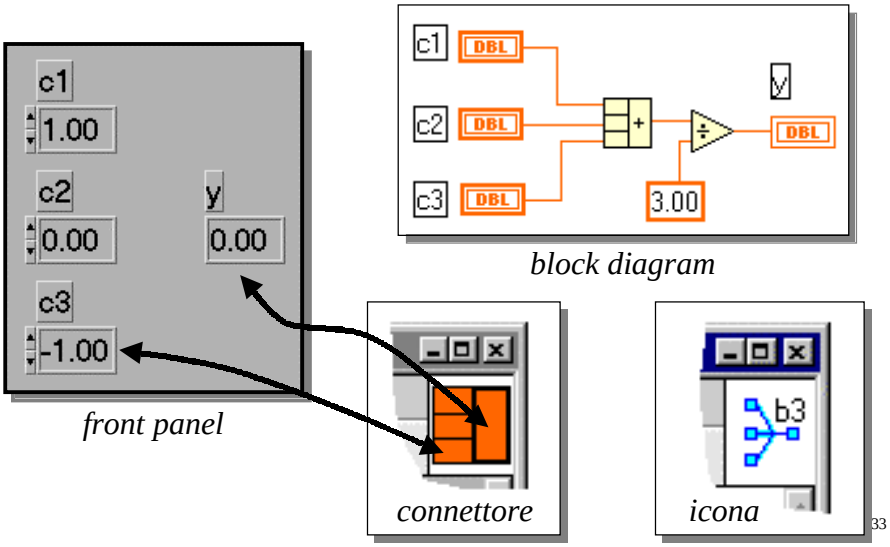
calcolo baricentro di un triangolo
coordinate su piano cartesiano XY

VI #1: baricentro rispetto a un asse, usato come sub-VI
VI #2: baricentro sul piano

32

Sub-VI: esempio

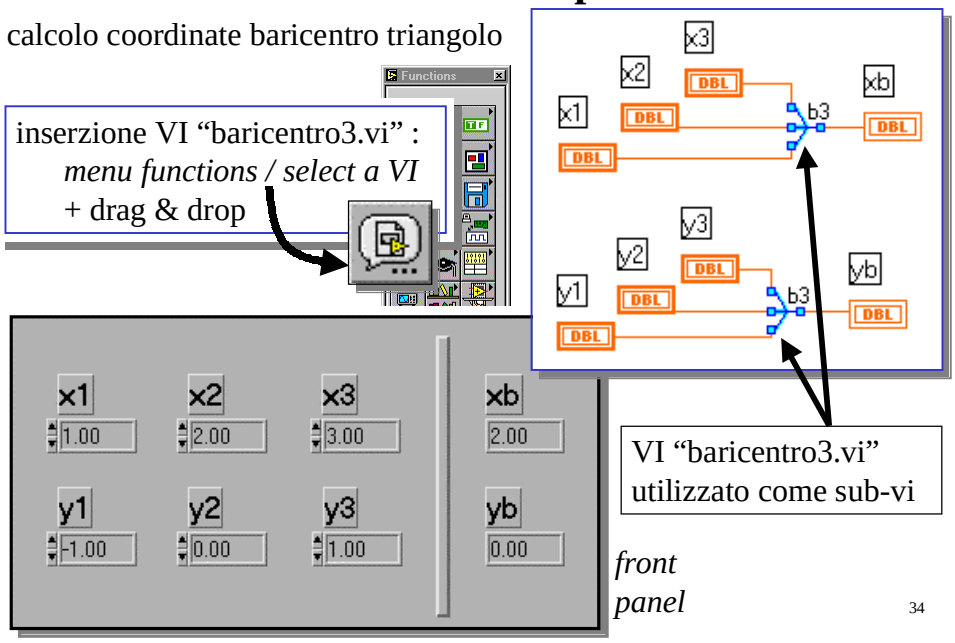
VI : calcolo baricentro lungo un asse coordinato (**`baricentro3.vi`**)



33

Sub-VI: esempio

calcolo coordinate baricentro triangolo



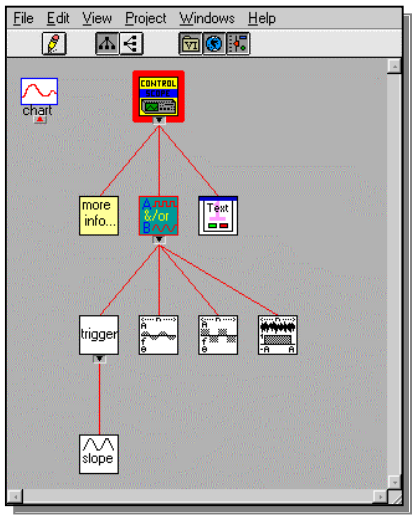
34

Sub-VI

programmazione a oggetti
- gerarchia
- ereditarietà
- polimorfismo

documentazione VI
- menu *window / show VI info*
- *popup, data operation / description*
- menu *project / show VI Hierarchy*

uso help contestuale !



esempio di gerarchia di VI

35

Controlli e indicatori

controlli (controls) = dato in ingresso
indicator (indicators) = dato in uscita

tipi di dato distinguibili da
- colore
- label icona

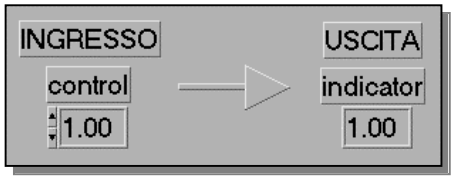
block diagram



controlli : bordo spesso

indicatori : bordo sottile

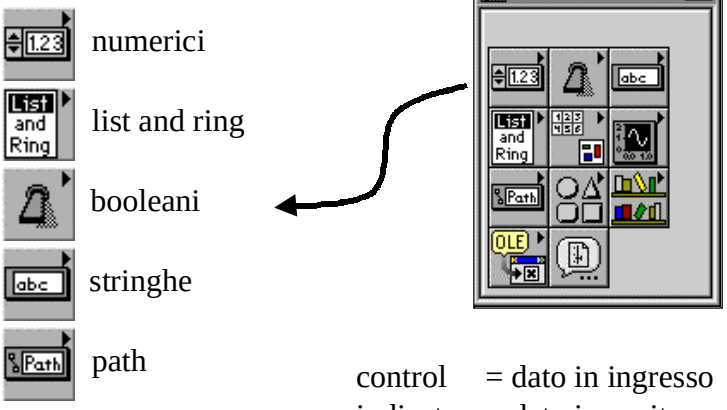
front panel



36

controlli e indicatori di base

tipi fondamentali di controlli e indicatori



control = dato in ingresso
indicator = dato in uscita

37

controlli e indicatori numerici

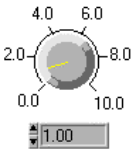


definiscono variabili di i/o di tipo numerico

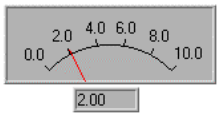
alcuni esempi



digital control



knob



meter

opzioni di popup (principali, valgono anche per altri tipi di controlli)

change to indicator (*control*) trasforma in indicatore (controllo)
format and precision: imposta num. decimali,…
show digital display: mostra finestra con valore numerico

data operations / make current value default imposta valore corrente come valore di default

38

list and ring



- particolare tipo di indicatore numerico che permette di associare a un numero (0, 1, 2, …) una stringa
  - usati come controlli forniscono in uscita numero corrispondente a opzione scelta
  - usati come indicatori mostrano opzione (stringa) corrispondente a numero di ingresso
  - compito programmatore gestire associazione numero - stringa in maniera corretta

- utili per gestione di liste del tipo menu, scelta tra alcuni elementi,…

- principali opzioni di popup
  show/digital display mostra numero selezionato
  add item after, add item before, remove item

39

controlli e indicatori booleani



possibili solo due stati (true/false)

controlli del tipo “interruttori”, indicatori del tipo “lampada spia”

azione meccanica
determina come funziona il controllo quando attivato (click)
esempi: cambia stato, cambia stato e ritorna stato originario, ...

esempi


led
button




round push button


vertical
toggle
switch

vertical switch

40

stringhe



contengono dati ascii

- opzioni di popup interessanti
  show scrollbar mostra barre di scorrimento
  code display interpreta caratteri preceduti da '\' (\00 - \FF = codice hex \b, \t, \n, ...

path



- impiegati per la gestione di percorsi su disco e nomi di file
- gestione indipendente dalla piattaforma (mac, win, unix,...)

41

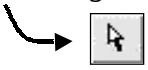
Strutture

Le *structures* (strutture) sono costrutti grafici che consentono di controllare il flusso di esecuzione di un VI



sono disponibili dalla *functions palette* quando è attiva la finestra *block diagram*

- per inserire una struttura nel *block diagram* eseguire drag & drop
- resize con *positioning tool*

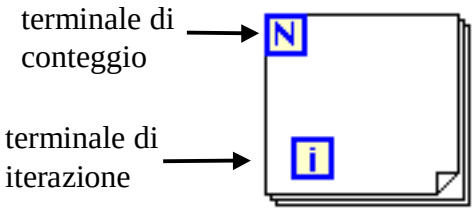


42

Strutture per esecuzione di cicli

for loop

esegue diagramma contenuto nel rettangolo che rappresenta la struttura “for loop” per $i = 0, \ldots, N-1$



pseudo-codice equivalente:

```
for i = 0 to N-1
   esegui sub-diagramma
```

43

Strutture per esecuzione di cicli

while loop

esegue diagramma contenuto nel rettangolo che rappresenta la struttura “while loop” finché il terminale di condizione non risulterà falso



pseudo-codice equivalente:

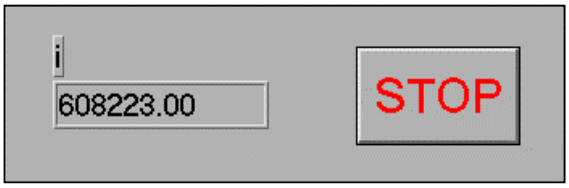
```
i = 0
do {
   esegui sub-diagramma
   i = i + 1
} while ”terminale di condizione” = true
```

44

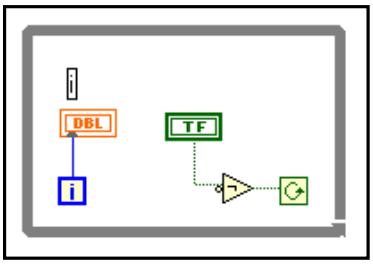
Strutture per esecuzione di cicli

un semplice esempio

incrementa contatore e mostra valore corrispondente finchè non viene premuto pulsante “stop”



front panel



block diagram

attenzione

- ingressi letti prima che inizi il ciclo
- uscite fornite al termine del ciclo

→ controlli e indicatori *interni* al ciclo

controllo “TF” fuori dal ciclo = ciclo infinito o eseguito una sola volta

45

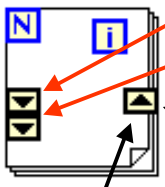
Strutture per esecuzione di cicli

registri a scorrimento (shift register)

- variabili locali che trasferiscono il valore dell’iterazione di un ciclo alla successiva

collegare da esterno valore di inzializzazione

qui disponibile valore della variabile “shift register” al passo precedente del ciclo due passi prima,...



collegare all’esterno valore assunto da var. a fine ciclo

qui passare valore alla variabile “shift register” che verrà resa disponibile al passo successivo del ciclo

46

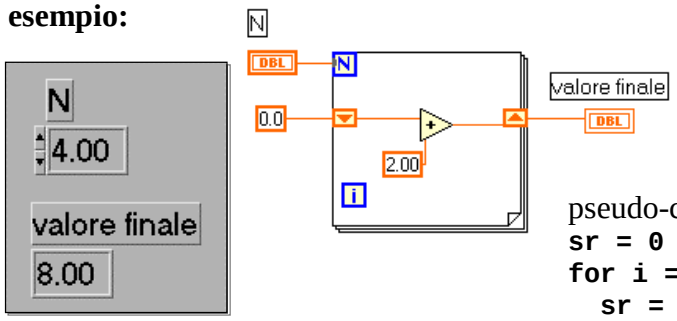
Strutture per esecuzione di cicli

registri a scorrimento (shift register)

- creazione shift register
  popup su bordo struttura “ciclo”
  add shift register aggiunge shift register
  add element aggiunge terminale sinistro

---

esempio:



pseudo-codice equivalente

```
sr = 0
for i = 0 to N-1
  sr = sr + 2
```

47

Variabili locali

possibile creare variabili locali al VI (menu functions / structure /local)

utili per la lettura all’interno di un ciclo del valore di un controllo *esterno* al ciclo

opzioni di popup
- associazione variabile locale a controllo
- impostazione variabile locale come sorgente di dati o destinazione dati

48

Variabili locali - esempio

si realizzi un VI che incrementa un contatore ogni 250 ms con possibilità di fermare sia l'iterazione, sia il VI



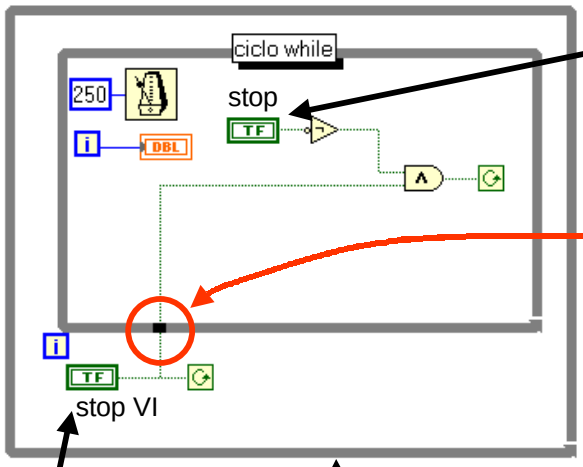
arresta iterazione ma non il VI (conteggio riparte da 0)

arresta VI (quindi anche iterazione) senza dover premere pusante "stop"

pannello frontale

49

esempio - una prima soluzione



arresta iterazione di incremento contatore

non funziona: il valore "stop VI" viene letto *una sola volta* all'inizio del ciclo più interno e *non* a ogni iterazione

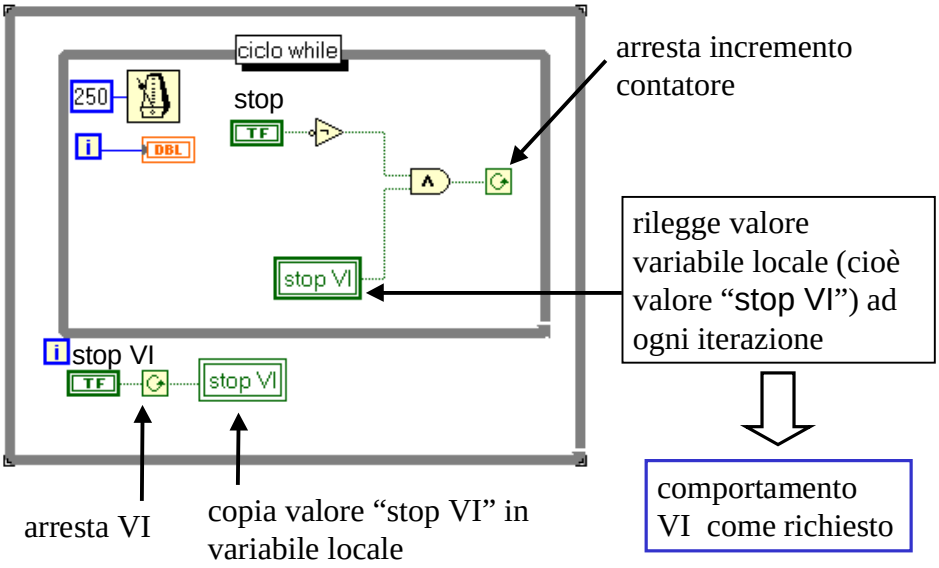
il VI si ferma *solo se* "stop VI" e "stop" sono stati attivati contemporaneamente

arresta VI

ciclo esterno: ripete tutte le istruzione del VI

50

esempio - soluzione con variabili locali



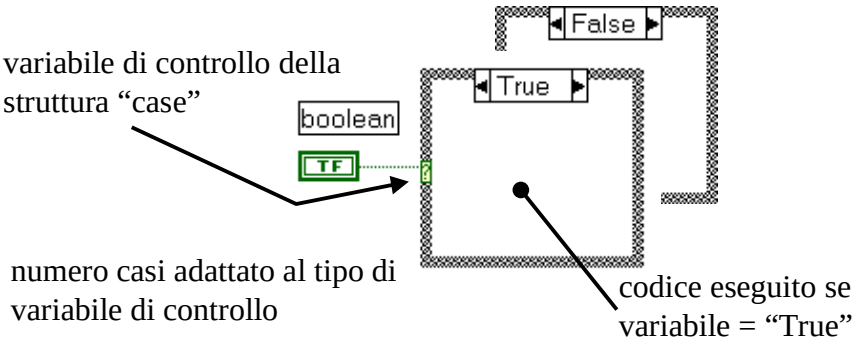
51

Strutture case

permettono la elaborazione condizionata di sub-diagrammi a seconda del valore di una variabile

popup su bordo struttura:

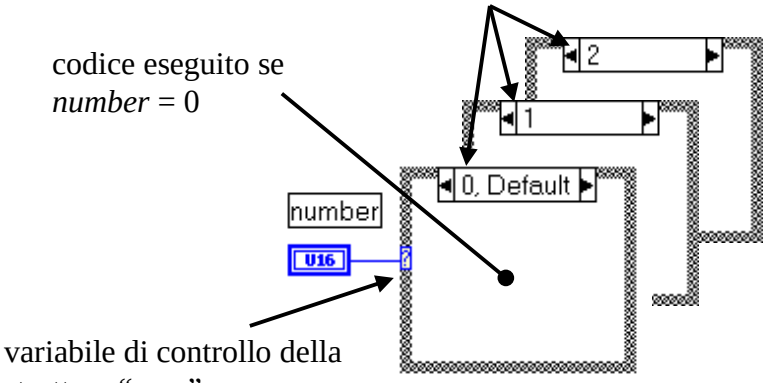
add case, remove case, duplicate case



52

Strutture case

variabile di controllo “case”di tipo numerico: possibilità di contemplare più di due casi



53

Strutture case - esempio

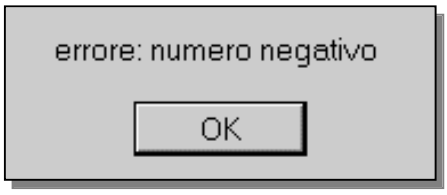
realizzare VI che permetta il calcolo della radice quadrata di un valore numerico con controllo sul segno dell’argomento

una possibile soluzione

pannello frontale



se il numero impostato, x, è negativo il VI produce un messaggio di errore

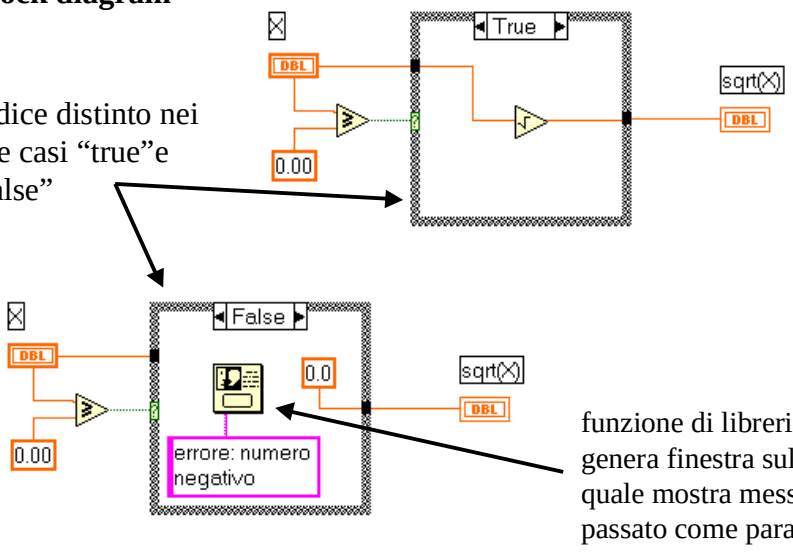


54

Strutture case - esempio

block diagram

codice distinto nei due casi “true”e “false”



funzione di libreria: genera finestra sulla quale mostra messaggio passato come parametro

55

Strutture sequence

- permettono la sequenzializzazione dei sub-diagrammi,contenuti nella struttura stessa
- ciascun sub-diagramma è definito da un “frame”
- per passare dati da un frame a un altro è necessario impiegare dei terminali di tipo “local”

opzioni di *popup*

add frame after, add frame before, duplicate frame, remove this frame, add sequence local, (remove)

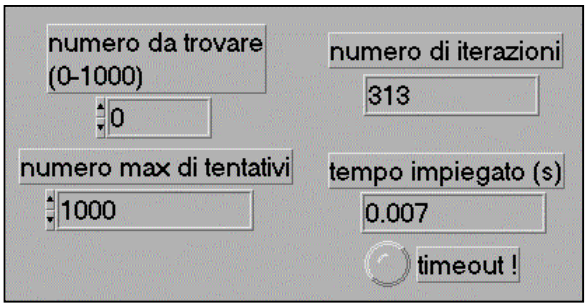
56

Strutture sequence - esempio

Realizzare VI che genera numeri casuali finchè il numero casuale corrente non è uguale a un valore preimpostato da pannello frontale. Il VI deve inoltre

- terminare l'elaborazione se sono stati generati *N* numeri (condizione di errore)
- calcolare il tempo totale di elaborazione

pannello frontale



57

Strutture sequence - esempio

block diagram

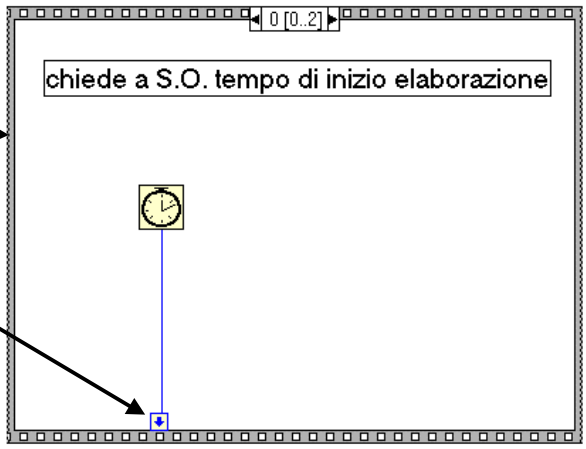
struttura "sequence" primo passo

indicatore numero di frame

indicatore di struttura *sequence*

terminale di variabile *local*

freccia rivolta verso bordo = dato passato agli altri frames



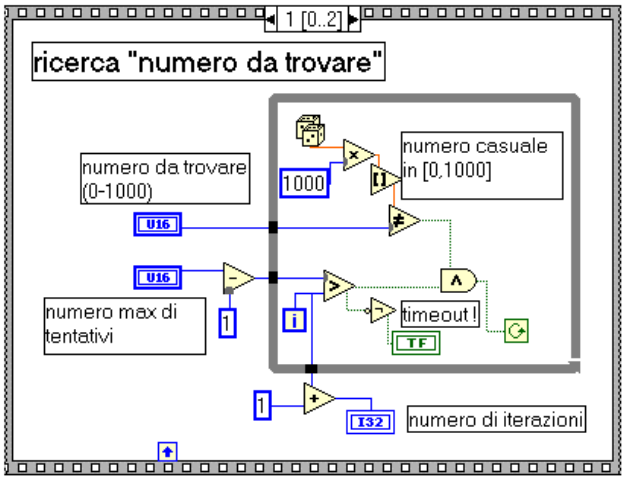
58

Strutture sequence - esempio

block diagram

struttura “sequence” secondo passo

- ciclo “while” per generazione numeri casuali
- confronto con dato fornito da pannello frontale
- controllo numero iterazioni



59

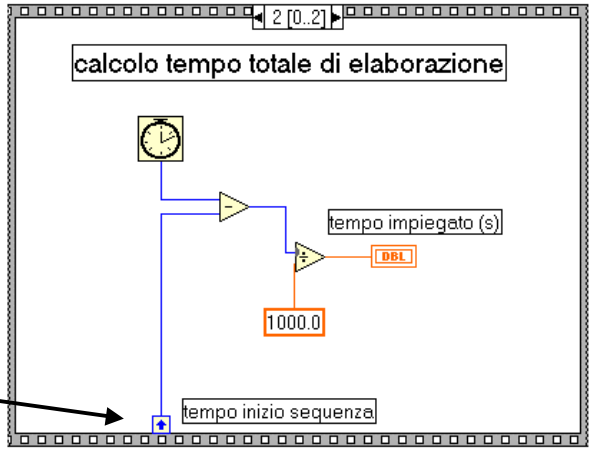
Strutture sequence - esempio

block diagram

struttura “sequence” terzo passo

- calcolo tempo di elaborazione

prelievo da terminale *local* valore del tempo di inizio elaborazione

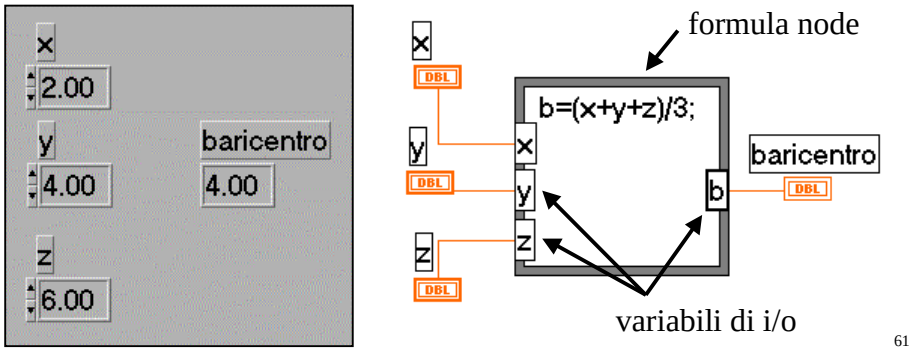


60

Struttura formula node

- permette di inserire formule algebriche
- formule separate da '**;**', più formule possibili
- variabili di i/o definite mediante *popup* su bordo della struttura

esempio: calcolo del baricentro di tre coordinate su di una stessa retta



61

Vettori e matrici

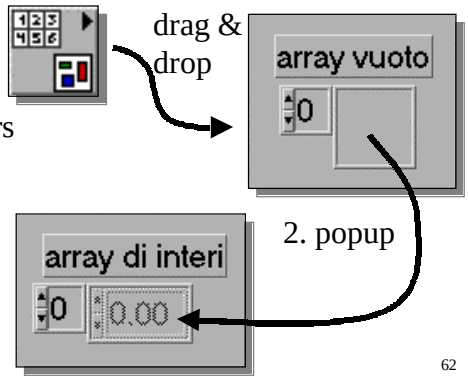
insiemi multidimesionali di dati dello stesso tipo, accessibili mediante indice

- vettori: lista monodimensionale
- matrici: liste multidimensionali (anche più di due indici)
- indici sempre a partire da 0

creazione di un array

1 drag & drop da menu controls / array & clusters

2 si trascina un elemento (o popup) nella finestra dell'array vuoto



62

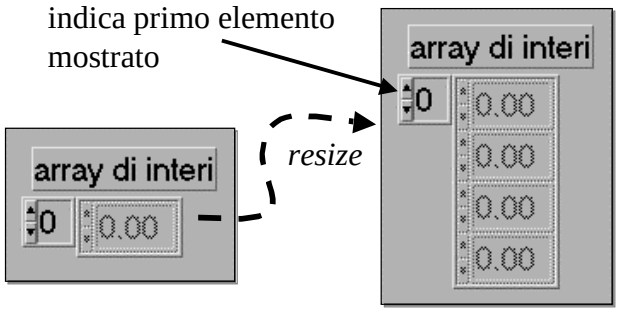
Vettori e matrici

creazione di un array

3 si ridimesiona la finestra associata all'array in modo da mostrare gli elementi desiderati

---

esempio



- *popup add dimension* aggiunge un indice al vettore → matrici

63

Vettori e matrici

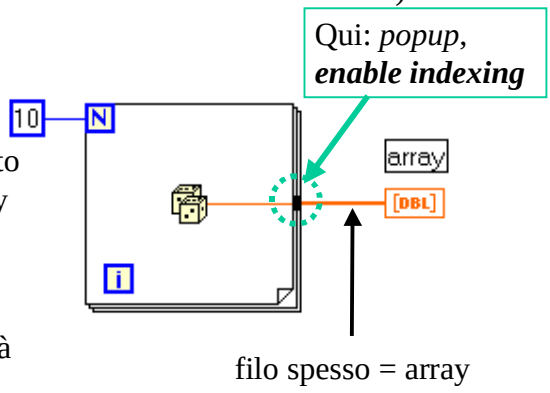
accesso a un array

- i cicli di default permettono accesso auto-indicizzato a un array
- **attenzione** eseguire popup su bordo struttura ciclo, ***enable indexing*** (in caso contrario *non* si ottiene alcun vettore)

esempio

collegamento di un elemento interno a un ciclo a un array esterno al ciclo stesso

al termine dell'esecuzione l'indicatore "array" conterrà 10 numeri causali

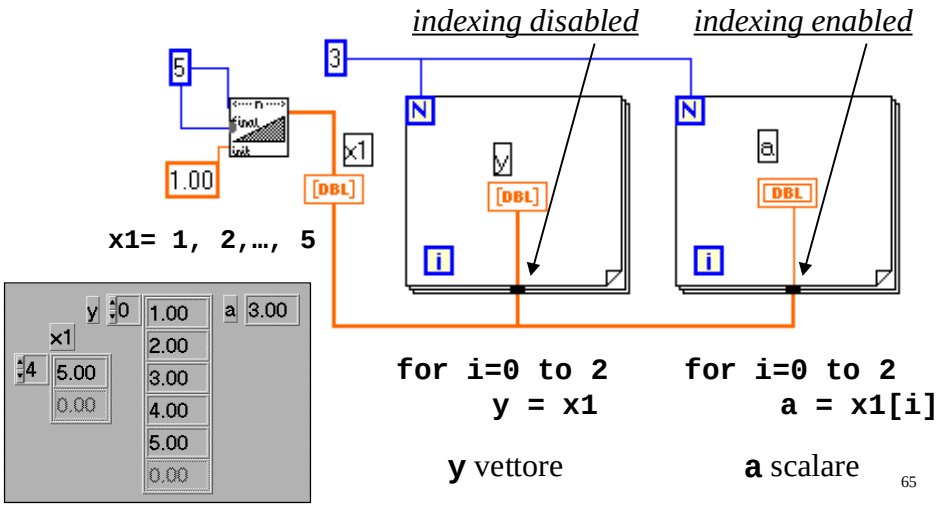


64

Vettori e matrici

accesso a un array

- esempio di uso della funzione di auto-indexing



65

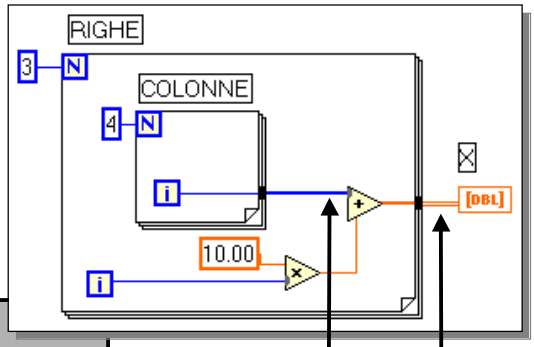
Vettori e matrici

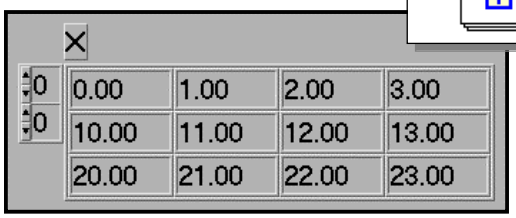
accesso a un array

- utilizzando cicli innestati è possibile gestire matrici

esempio

```
for row = 0 to 2
 for col = 0 to 3
   x[row,col]=col+row*10
```





filo spesso

filo “doppio”

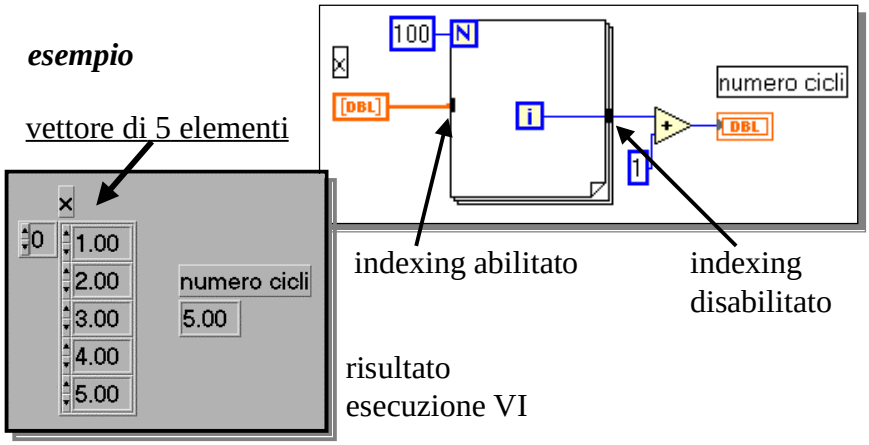
66

Vettori e matrici

accesso a un array

- la connessione di un array a un ciclo "*for loop*" forza il numero di iterazioni alla dimensione dell'array, se l'autoindicizzazione è abilitata

esempio



67

Vettori e matrici

polimorfismo

- le funzioni aritmetiche disponibili in libreria sono polimorfiche, nel senso della programmazione a oggetti. Quindi, per definizione, possono accettare in ingresso tipi di dati distinti e sono in grado di costruire un uscita coerente

esempi

- somma di uno scalare con un vettore = somma lo scalare a tutti gli elementi del vettore
- somma tra due vettori con le stesse dimensioni = fornisce un vettore i cui elementi sono la somma di elementi corrispondenti dei due vettori di ingresso

68

Vettori e matrici

principali funzioni di libreria

- *initialize array*
  assegna valore iniziale a elementi di un array
- *array size*
  fornisce un veottore contenente le dimesioni della matrice
- *build array*
  costruisce una matrice (VI "*resizable*" !)
- *array subset*
  estrae sotto-matrice
- *index array*
  estrae un elemento da un vettore

69

Cluster

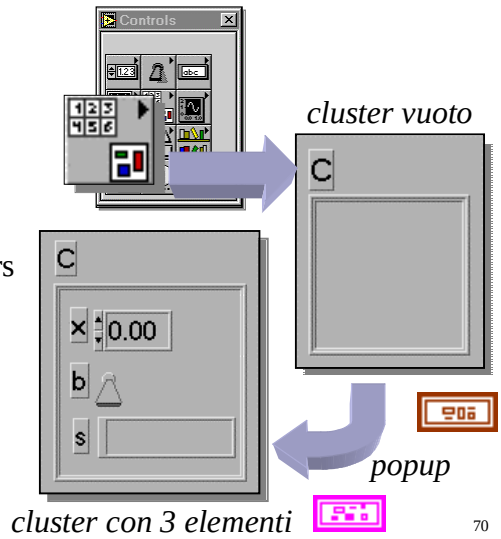
strutture che raggruppano più dati anche di tipo diverso

costruzione di VI più ordinati

creazione di un cluster

1 drag & drop da menu controls / array & clusters

2 si inseriscono (popup) nella finestra del cluster gli elementi che lo dovranno comporre



70

Cluster

ordine di un cluster

- elementi di un cluster logicamente ordinati (0,1…); ordinamento stabilito per ordine di creazione;
- possibilità di modifica ordine (popup/cluster order)

funzioni interessanti

- *bundle* riunisce più collegamenti in un cluster
- *unbundle* trasforma un cluster in collegamenti singoli,

  nota ordine di scomposizione determinato da ordine logico del cluster, attenzione a non invertire variabili !

71

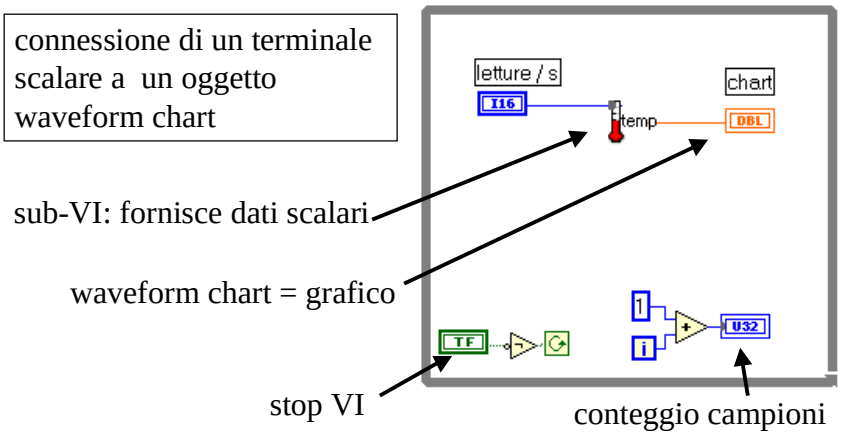
Diagrammi

collezione di oggetti adatti alla rappresentazione di grafici

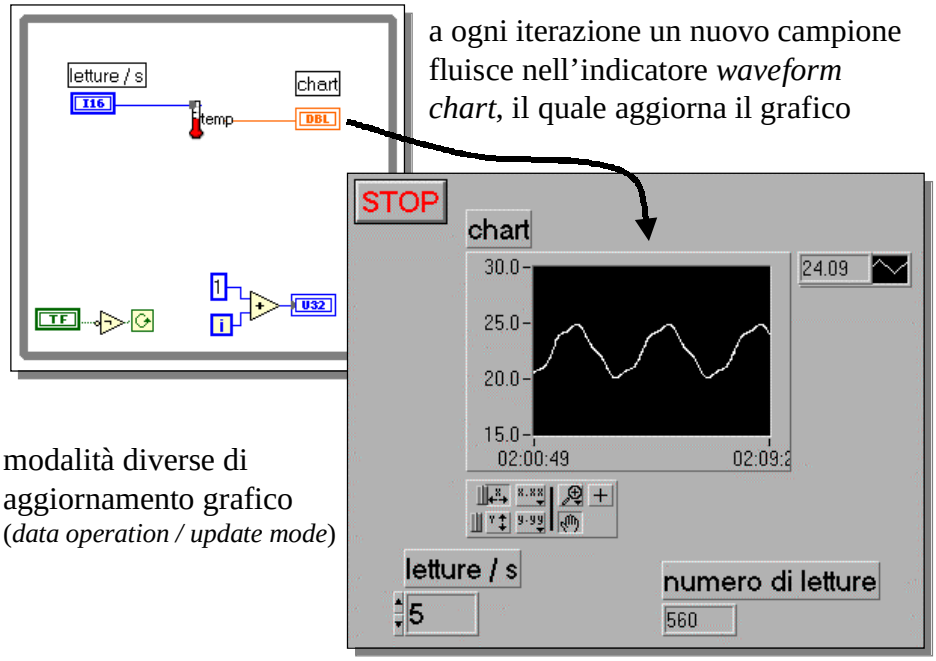
diverse modalità di connessione ai terminali interni al VI

---

connessione di un terminale scalare a un oggetto waveform chart



72



a ogni iterazione un nuovo campione fluisce nell'indicatore *waveform chart*, il quale aggiorna il grafico

modalità diverse di aggiornamento grafico (*data operation / update mode*)

73

Diagrammi

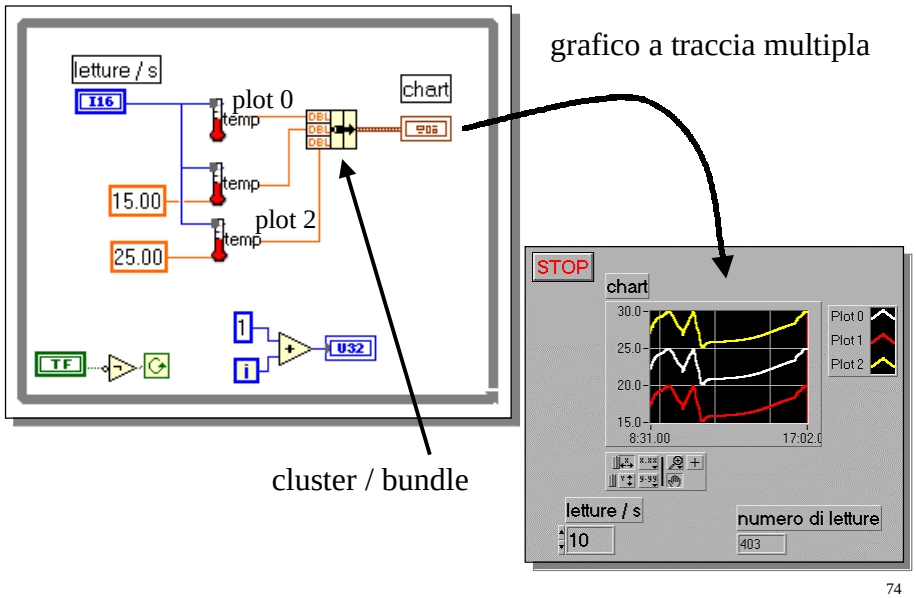


grafico a traccia multipla

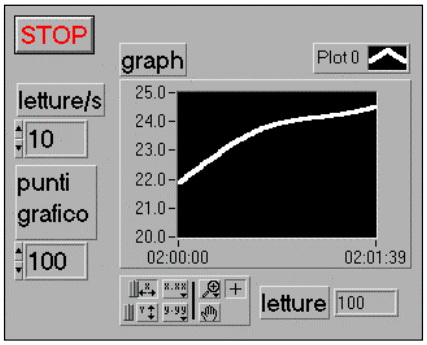
cluster / bundle

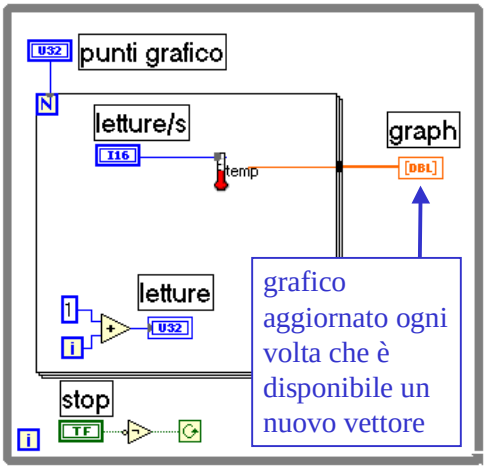
74

Grafici

- Aspetto su pannello frontale identico a diagrammi
- Differiscono dai diagrammi in quanto costruiti in un unico momento a partire da un vettore

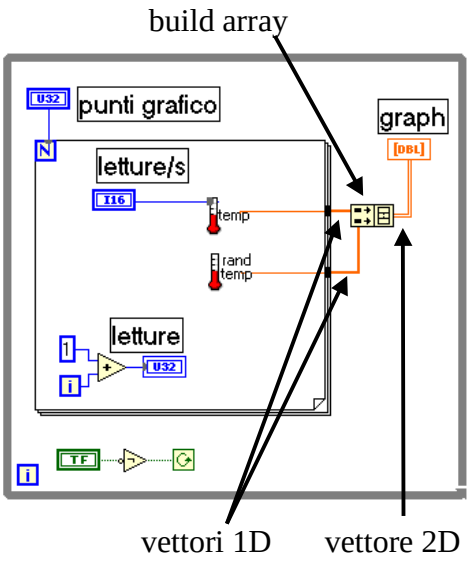
esempio



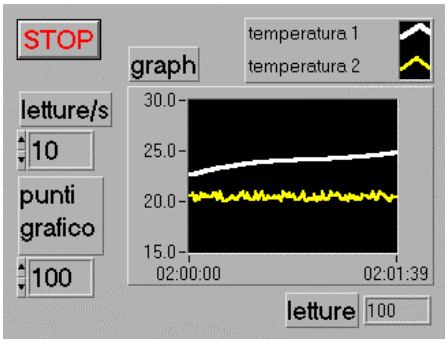


75

Grafici



analogamente al caso dei diagrammi, è possibile costruire grafici con più serie di dati utilizzando la funzione *build array*

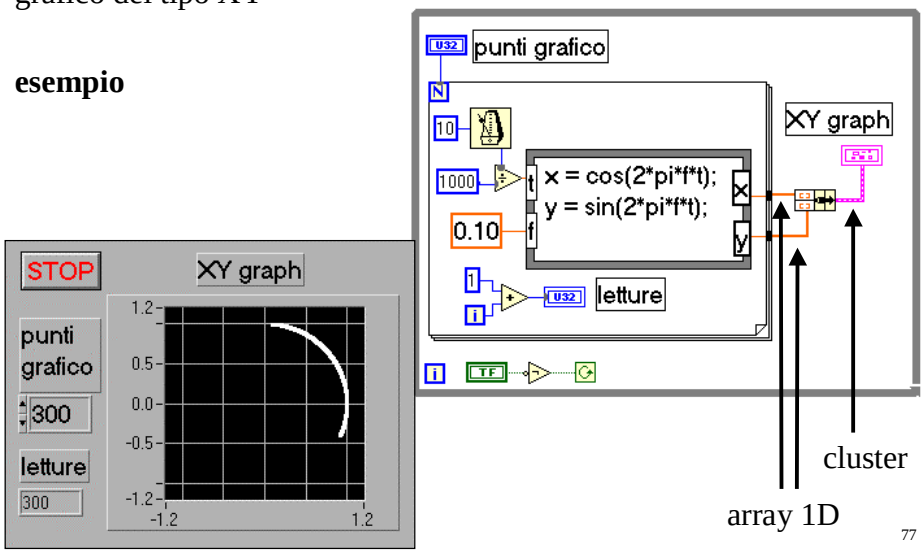


76

Grafici

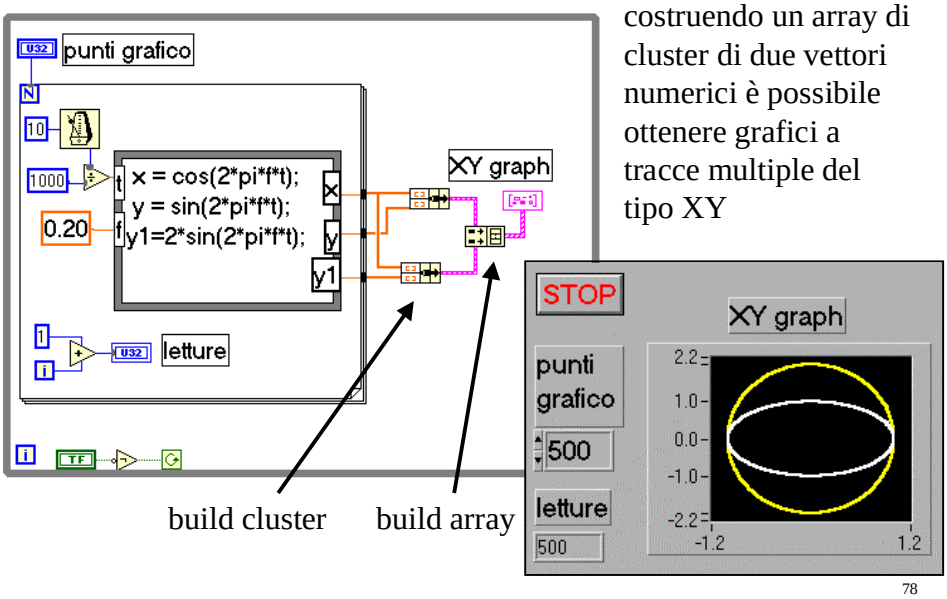
costruendo un cluster di due vettori numerici è possibile ottenere un grafico del tipo XY

esempio



77

Grafici



costruendo un array di cluster di due vettori numerici è possibile ottenere grafici a tracce multiple del tipo XY

78

Stringhe

stringa: insieme di caratteri ascii

importanza della manipolazione di stringhe

- la strumentazione riceve comandi e fornisce risposte nella forma di stringhe,
- necessità di combinare informazioni lette da pannello frontale in una “stringa di comando” da inviare alla strumentazione
- necessità di estrarre da una stringa informazioni utili all’aggirnamento degli indicatori presenti su pannello frontale

esempio

lettura controllo “V/div”, → impostazione configurazione canale oscilloscopio

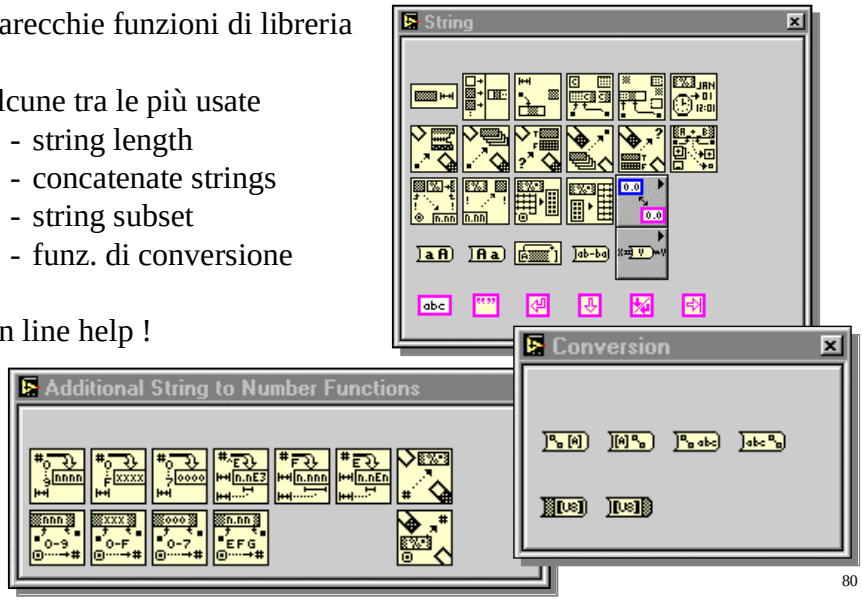
79

Stringhe

• parecchie funzioni di libreria

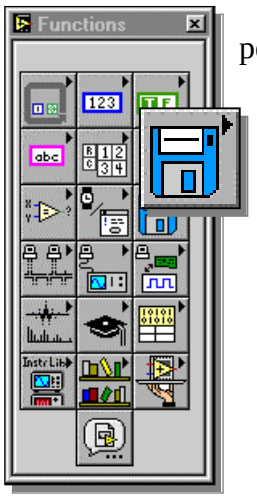
• alcune tra le più usate
  - string length
  - concatenate strings
  - string subset
  - funz. di conversione

• on line help !



80

file i/o



permettono la lettura e il salvataggio di dati da disco

palette functions
file i/o

come funzionano in genere le funzioni di file i/o

- si aspettano come input il nome di un file
- file testo
- aprono il file, eseguono le operazioni utili (lettura/scrittura…), chiudono il file

alcune funzioni interessanti

read (write) characters from (to) file,
read (write) lines from (to) file
read (write) from (to) spreadsheet file

81

Configurazione dell'aspetto di un VI

popup su icona / VI setup

execution options

- *show front panel when loaded* se attiva LV mostra pannello frontale quando VI caricato in memoria
- *show front panel when called*, se attiva il pannello frontale viene aperto quando il VI viene richiamato come sub-VI
- *close afterwards if originally closed*, chiude front panel quando sub-VI termina esecuzione
- *suspend when called*, si ferma l'esecuzione appena VI invocato (= breakpoint)

window options

configura aspetto finestra al momento esecuzione (scrollbar, toolbars,…)

82

